

# Five functions, four lines or fewer

Sixteen minutes, in the room, finished before you leave. Each function is a variation on something from the last hour. If you are stuck for three minutes, put your hand up — that is what the time is for.

Guided lab

16 minutes

Not graded

Not a prerequisite for session 5

[Before you start](#)[1 · convert](#)[2 · is\\_even](#)[3 · bigger](#)[4 · total](#)[5 · average](#)[Check yourself](#)[If you finish early](#)

## Before you start

Open the provided `lab01.ipynb`. Each function has its own cell, with the test lines already underneath it. Fill in the function, run the cell. If the printed answer matches and the assertions pass silently, move on.



### How to read the tasks

Each one gives you the **name** the function must have, what it **receives**, what it must **return**, and a line to test it with. The name matters — later tasks call the earlier functions. Nothing here needs more than four lines inside the function.



### Type it, do not paste it

You will be tempted to copy the examples from the lecture. Do not. The typing is the lab. Your hands are learning where the colons and the indentation go, and that knowledge is what stops you losing twenty minutes to a syntax error in session 8.

#### TASK 1 · 2 MINUTES

## convert\_temperature

One argument, one calculation, one return. The warm-up.

1

**Receives:** a temperature in Celsius. **Returns:** the same temperature in Fahrenheit.

The formula is  $F = C \times 9/5 + 32$ .

test it with this

PYTHON

```
print(convert_temperature(100))    # should print 212.0
print(convert_temperature(0))      # should print 32.0
```



### If it prints 212 instead of 212.0

You have written `9//5` somewhere instead of `9/5`. Both run. Only one is right, and the difference is the trap from part 2 of the lecture.

#### TASK 2 · 3 MINUTES

## is\_even

A condition, and a function that returns something other than a number.

2

**Receives:** a whole number. **Returns:** `True` if it is even, `False` if it is not.

A number is even when the remainder after dividing by 2 is zero. The remainder operator is `%`.

test it with this

PYTHON

```
print(is_even(10))    # True
print(is_even(7))     # False
print(is_even(0))     # True – zero is even
```

The shape you want is an `if` that returns one thing and an ordinary `return` for everything else:

PYTHON

```
def is_even(n):  
    if n % 2 == 0:  
        return True  
    return False
```

Once it works, notice that `return n % 2 == 0` does the whole job in one line, because a comparison already produces `True` or `False`. Both are correct; the second is what you will see in real code.

### TASK 3 · 3 MINUTES

## bigger

Two arguments. You wrote this on paper in the lecture — now make it run.

3

**Receives:** two numbers. **Returns:** the larger of the two. If they are equal, returning either is fine.

test it with this

PYTHON

```
print(bigger(3, 9))      # 9  
print(bigger(-2, -7))   # -2 ← check this one carefully
```

### The negative case is the point of this task

A version that starts from zero and compares upward gets the first test right and the second one wrong. If your function returns 0 or -7 for the second line, you have found — on purpose — the same bug that was planted in the diagnostic.

#### TASK 4 · 4 MINUTES

## total

The accumulator. This is the task that matters.

# 4

**Receives:** a list of numbers. **Returns:** their sum, computed with a loop.



**Do not use** `sum()`

Python has a built-in that does this in one word, and from next week you may use it freely. Today the loop *is* the exercise — it is the shape of every training loop you will write from session 9 onward, and writing it once by hand is what makes that later code readable.

test it with this

PYTHON

```
print(total([70, 85, 60, 95])) # 310
print(total([5]))               # 5
print(total([]))               # 0 - an empty list sums to nothing
```

Three lines, in this order: the accumulator before the loop, the update inside it, the return after it.

PYTHON

```
def total(values):
    running = 0
    for v in values:
        running = running + v
    return running
```

If your third test printed something other than 0, check where your `return` is indented. If it sits inside the loop, an empty list never reaches it at all.

## average

Everything at once, plus one case that has to be handled before it happens.

5

**Receives:** a list of numbers. **Returns:** their mean. If the list is empty, return 0 rather than crashing.

You may call `total` from task 4 — that is the point of giving the functions names. `len(values)` gives you how many items there are.

test it with this

PYTHON

```
print(average([70, 85, 60, 95])) # 77.5
print(average([10, 20]))         # 15.0
print(average([]))               # 0 — and no error
```



### This is the function from the lecture opening

If task 5 runs and all three lines print what they should, you have met today's target exactly. That is the whole of what session 4 was for.

## Check yourself before you leave

Run this in a new cell. It uses all five functions at once and prints a single line.

the whole lab, in one cell

PYTHON

```
readings = [21.5, 23.0, 19.5, 24.0]

print("mean °C:", average(readings))
print("mean °F:", convert_temperature(average(readings)))
print("warmest:", bigger(readings[0], readings[3]))
print("count is even:", is_even(len(readings)))
```

```
mean °C: 22.0
mean °F: 71.6
warmest: 24.0
count is even: True
```

If `mean °F` came out as 71.60000000000001, nothing is wrong. Floats are stored in binary and 0.6 has no exact binary form, so tiny residues appear. You will meet this properly in session 6; for now, ignore it.

## If you finish early

Unmarked, and not needed for Tuesday. These are for anyone who found the five tasks quick.

- A** `count_above(values, threshold)` — how many numbers in the list are strictly greater than the threshold. Test: `count_above([3, 9, 4, 12, 1], 5)` gives 2. The trap is hard-coding 5 inside the function instead of using the argument.
- 
- B** `largest(values)` — the largest number, without using `max`. Make it work for `[-5, -2, -9]`, which is where the obvious version fails.
- 
- C** `running_totals(values)` — return a list of the running sums, so `[1, 2, 3]` gives `[1, 3, 6]`. You need one thing you have not met yet: `result.append(x)` adds `x` to the end of a list. This is a preview of Tuesday.
- 
- D** `threshold(scores, t)` — turn a list of scores into a list of 0/1 predictions: 1 when a score is at least `t`, 0 otherwise. `threshold([0.2, 0.5, 0.9], 0.5)` gives `[0, 1, 1]`. Check the score that equals `t` exactly — that is where `>` and `≥` disagree. This is, quite literally, the last line of every classifier you will build from session 11.



### Or better: help the person next to you

If you have finished and someone nearby has not, explaining task 4 to them will teach you more than task C will. It is also the single most useful thing you can do for this class, and I will notice.